

Chapter 20

Agent Design Patterns

Building effective agents requires more than a powerful model and a set of tools. The *architecture*—how the LLM is orchestrated, how tasks are decomposed, and how control flows between components—determines whether an agent is reliable, debuggable, and cost-effective. This chapter presents the canonical design patterns that have emerged from production deployments at Anthropic, OpenAI, Google, and the open-source community.

When to Use Agents vs. Workflows

Not every task requires an autonomous agent. The key distinction:

- **Workflows:** Predefined control flow, LLM calls at specific steps. Predictable, testable, cheaper. Use when the task structure is known.
- **Agents:** LLM dynamically decides what to do next. Flexible, handles novel situations. Use when tasks require adaptive decision-making.

Start with workflows. Graduate to agents only when the task genuinely requires dynamic routing or open-ended exploration.

20.1 Workflow Patterns

These patterns—adapted from Anthropic’s taxonomy of agentic building blocks [10]—use LLMs within a *predefined* control flow. The system (not the model) decides the execution order.

20.1.1 Prompt Chaining

The simplest pattern: break a complex task into a fixed sequence of LLM calls, piping the result of one call as context into the next. Validation gates between steps catch errors early before they propagate downstream.

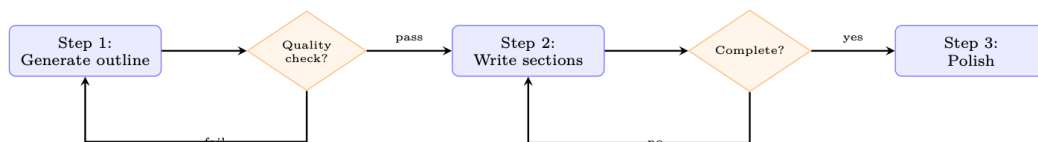


Figure 20.1: Prompt chaining with quality gates. Each step is a separate LLM call. Gates can be LLM-based or programmatic.

When to use: Tasks that are naturally sequential—content generation, data transformation, multi-stage analysis.

Key advantage: Each step can use a different prompt, model, or temperature. Intermediate results are inspectable and debuggable.

20.1.2 Routing

A classifier (LLM or traditional) examines the input and dispatches to a specialized handler.

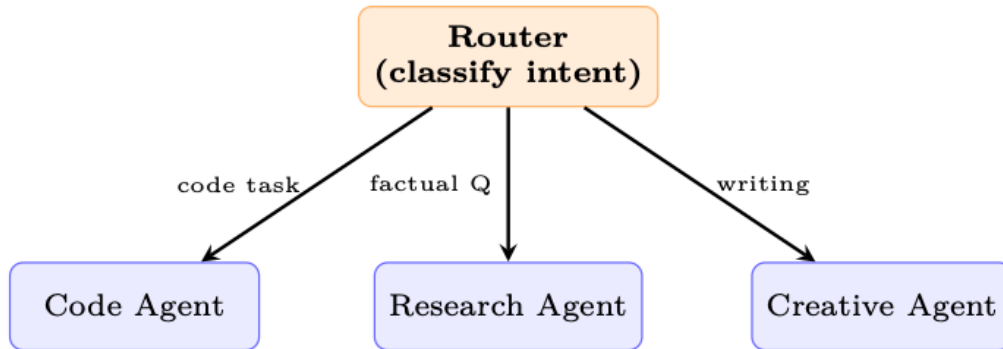


Figure 20.2: Routing pattern: input is classified once, then handled by a specialist.

When to use: Distinct task types with different optimal prompts, tools, or models. Customer support triage, multi-modal input handling.

20.1.3 Parallelization

Multiple LLM calls run concurrently, with a programmatic layer combining their outputs. Two sub-patterns emerge:

- **Sectioning (fan-out):** Partition the input into disjoint chunks and process each independently—e.g., run security, performance, and style checks on a codebase simultaneously.
- **Voting (redundancy):** Issue the same prompt N times with different seeds or temperatures, then select the best result via majority vote [368], reward-model scoring, or LLM-as-judge.

Parallelization Example: Code Review

1. **Parallel calls:** Security review || Performance review || Style review
2. **Aggregation:** Merge all findings, deduplicate, rank by severity

Latency = $\max(\text{individual calls})$ rather than $\sum(\text{individual calls})$.

20.1.4 Orchestrator-Workers

Here the LLM itself decides how to split the work. An orchestrator model analyzes the task, produces a plan of subtasks, dispatches each subtask to a worker LLM (potentially with different prompts or tools), and finally merges their outputs into a coherent result. The key difference from parallelization is that the decomposition logic is model-generated, not hard-coded.

When to use: Open-ended problems where the number and nature of subtasks cannot be enumerated at design time—e.g., “refactor this codebase” requires first understanding the dependency graph before deciding which files to modify.

20.1.5 Evaluator-Optimizer

A two-model feedback loop [246]: a generator produces candidate outputs while a separate evaluator scores them against explicit criteria. If the score falls below a threshold, the evaluator’s critique is appended to the generator’s context and the cycle repeats until the quality bar is met or a retry budget is exhausted.

When to use: Tasks with clear quality criteria—code that must pass tests, translations that must preserve meaning, writing that must match a style guide.

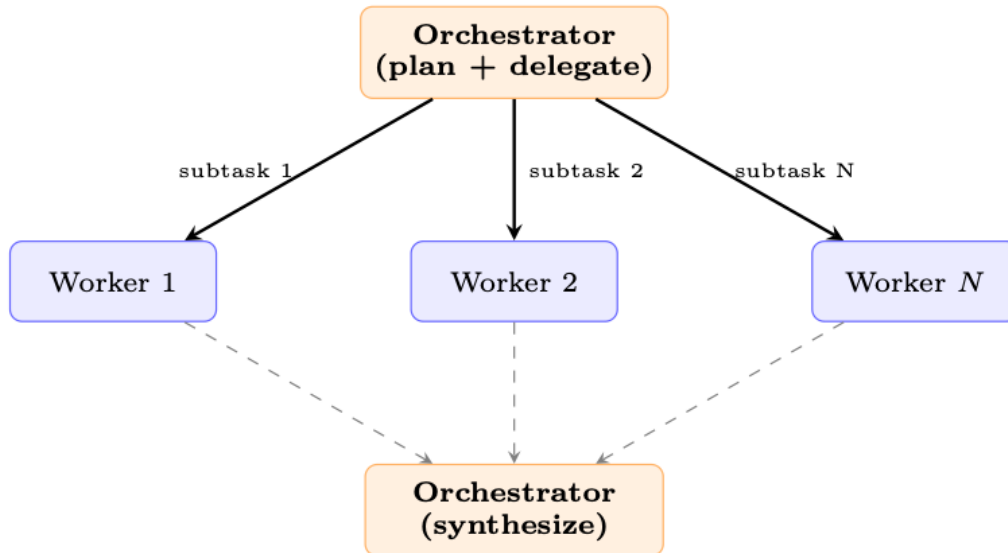


Figure 20.3: Orchestrator-workers: the LLM decides how to decompose the task and synthesizes worker results.

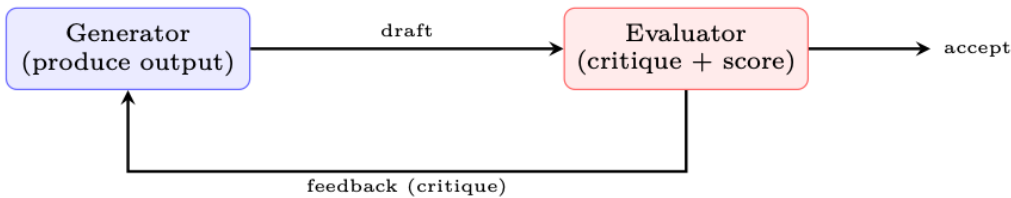


Figure 20.4: Evaluator-optimizer: iterative refinement without training.

20.2 Autonomous Agent Patterns

These patterns give the LLM control over the execution flow itself.

20.2.1 ReAct (Reason + Act)

The foundational agent pattern [408]. The LLM alternates between thinking (internal reasoning), acting (tool calls), and observing (processing results) in a loop until it produces a final answer.

ReAct Implementation Essentials

- **Scratchpad:** The “Thought” step is logged but not shown to the user.
- **Tool parsing:** The harness extracts structured tool calls from model output.
- **Max iterations:** Always cap the loop (typical: 10–25 iterations).
- **Termination:** Model outputs a special action (e.g., `final_answer`) or no tool call is detected.

20.2.2 Planning Agents

The agent generates an explicit plan before executing, and can revise the plan mid-execution [364].

Planning Agent: Research Report Generation

User request: “Write a 2-page report comparing transformer architectures for time-series forecasting.”

Step 1 — Plan generation (single LLM call):

Table 20.1: Planning strategies compared

Strategy	Replanning	Characteristics
Plan-then-Execute	Never	Simple; fragile to unexpected results
Adaptive	On failure	Replans only when a step fails; moderate cost
Continuous	Every step	Full re-evaluation after each observation; expensive but robust
Hierarchical	On sub-plan done	High-level plan fixed; sub-plans generated dynamically

```

plan = [
  {"id": 1, "task": "Search for recent transformer-based "
                  "time-series models (2023-2025)",
   "tool": "search_web", "deps": []},
  {"id": 2, "task": "Read top 5 papers, extract key methods",
   "tool": "read_papers", "deps": [1]},
  {"id": 3, "task": "Build comparison table (architecture, "
                  "dataset, metrics)",
   "tool": "none", "deps": [2]},
  {"id": 4, "task": "Write introduction + methodology section",
   "tool": "none", "deps": [2]},
  {"id": 5, "task": "Write results + conclusion",
   "tool": "none", "deps": [3, 4]},
  {"id": 6, "task": "Review and polish final report",
   "tool": "none", "deps": [5]},
]

```

Step 2 — Execution with adaptive replanning: The agent executes steps in dependency order. After step 1, the search returns only 3 relevant papers. The agent *replans*: it adds a sub-step to broaden the search to adjacent domains (e.g., PatchTST, iTransformer). The revised plan continues from step 2 with the expanded corpus.

Key insight: The plan is a *living document*—it provides structure but adapts to observations. The harness tracks dependencies as a DAG and only executes steps whose predecessors have completed.

20.2.3 Reflection and Self-Critique

The agent pauses to evaluate its own trajectory and correct course:

1. **Output validation:** “Is this correct? Did I miss anything?”
2. **Trajectory review:** Review last k steps, identify mistakes or inefficiencies.
3. **Strategy revision:** Reconsider the overall approach (“Am I solving the right problem?”).

Reflexion: Learning from Failure

The **Reflexion** pattern [326] maintains a persistent “reflection memory.” After each failed attempt, the agent writes a natural-language reflection (“I failed because I didn’t check the edge case”). On the next attempt, these reflections are included in the prompt—enabling learning across episodes without weight updates.

20.2.4 Tool-Use Patterns

How an agent invokes tools significantly affects its reliability, latency, and cost. Five canonical patterns have emerged [315]:

Single-Turn Tool Use. The simplest pattern: the model issues one tool call, receives the result, and produces a final answer. Sufficient for factual lookups, unit conversions, or single API queries. The harness makes exactly two LLM calls (one to decide on the tool, one to synthesize the result).

Table 20.2: Tool invocation patterns

Pattern	Description	Example
Single-turn	One tool call per LLM response	Simple Q&A with search
Multi-tool	Multiple parallel tool calls in one response	Search + calculate + format
Sequential	Tool output feeds into next tool call	Search → read → extract
Nested	Tool call triggers another agent	Code agent calls test-runner
Fallback	Preferred tool fails; try alternative	API → scrape → cache

Multi-Tool (Parallel). Modern APIs (OpenAI, Anthropic) allow the model to request multiple tool calls in a single response. The harness executes them concurrently and returns all results together. This dramatically reduces latency for tasks requiring independent information from multiple sources—e.g., fetching stock price, weather, and calendar simultaneously. The key constraint: the tools must be *independent* (no tool’s output is needed as input to another).

Sequential (Pipeline). Each tool’s output feeds into the next tool’s input, forming a data pipeline. The model decides the next tool based on the previous result. Common in research workflows: `search` → `fetch_page` → `extract_data` → `analyze`. The harness must track the growing context and may need to summarize intermediate results to stay within budget.

Nested (Agent-as-Tool). A tool call invokes an entirely separate agent—with its own prompt, tools, and context. The parent agent treats the sub-agent as a black-box function. This enables specialization: a research agent delegates code execution to a coding agent, which has access to a sandbox and test runner. The Swarm pattern [276] generalizes this via handoffs between specialized agents.

Fallback (Graceful Degradation). The harness tries tools in priority order: if the preferred tool fails (timeout, rate limit, API error), it automatically falls back to an alternative. The model need not be aware of the fallback logic—the harness handles it transparently. Example: primary search API → backup search → cached results → inform model that search is unavailable.

20.3 Design Principles

The following principles, distilled from Anthropic’s guide to building effective agents [10], apply across all patterns:

1. **Keep it simple.** Use the simplest architecture that works. Add complexity only when demonstrated necessary. A prompt chain that solves the problem is always preferable to a multi-agent system that might.
2. **Transparency over cleverness.** Every step should be inspectable. Avoid hidden state or implicit reasoning. When an agent fails, you need to understand *why*—opaque architectures make debugging impossible.
3. **Provide good tools.** Well-documented, well-typed tools with clear error messages are force multipliers. A tool with a vague description will be misused; a tool with a precise schema and usage guidance will be selected correctly.
4. **Plan for failure.** Every tool call can fail. Build retry logic, fallbacks, and graceful degradation at the harness level so the model does not need to reason about infrastructure failures.

5. **Use structured outputs.** Constrained generation (JSON schema, function calling) prevents parse failures. An agent that produces free-form text requiring regex parsing is fragile; one that produces validated JSON is robust.
6. **Test with diverse inputs.** Agent behaviour is more variable than single-turn chat. The same prompt can produce different tool-call sequences on different runs. Test adversarially, with edge cases, ambiguous requests, and malformed inputs.

20.4 Pattern Selection Guide

Choosing the right pattern depends on three factors: (1) how predictable the task structure is, (2) how many LLM calls you can afford in latency and cost, and (3) whether quality requires iteration. Use the table below as a decision matrix—start from the top (simplest) and move down only when the simpler pattern demonstrably fails.

Table 20.3: When to use each agent design pattern

Pattern	Complexity	LLM Calls	Best For
Prompt chaining	Low	N (fixed)	Sequential tasks, content pipelines
Routing	Low	$1 + 1$	Multi-type inputs, triage
Parallelization	Low	N (parallel)	Independent subtasks, voting
Orchestrator-workers	Medium	Variable	Unknown decomposition
Evaluator-optimizer	Medium	2–10 (loop)	Quality-critical outputs
ReAct	Medium	3–25 (loop)	General tool-use, exploration
Planning agent	High	5–50+	Long-horizon, multi-step tasks
Reflection	High	+50% overhead	Tasks where first attempt often fails
Multi-agent	High	Many	Complex domains, specialization

Patterns are composable: a planning agent may use prompt chaining for individual steps, an evaluator-optimizer within its review phase, and routing to dispatch subtasks to specialists. The art is knowing when to stop adding layers.